Tutorial 2 (to be completed in week2)
Data Modeling (Part 2) & Relational Set Operators

Tutorial Objectives:

- Transform conceptual model into a logical model using Visio.
- Learn about relational set operators.

Tutorial Tasks:

a. Arrive at a logical model by mapping the conceptual model (Tiny College example) to the chosen relational constructs (tables, primary and foreign keys, etc.).
b. Apply relational set operators in Oracle 12c for a given problem domain. Initial SQL script file is provided.
c. Analyse some relational operators using the script provided.

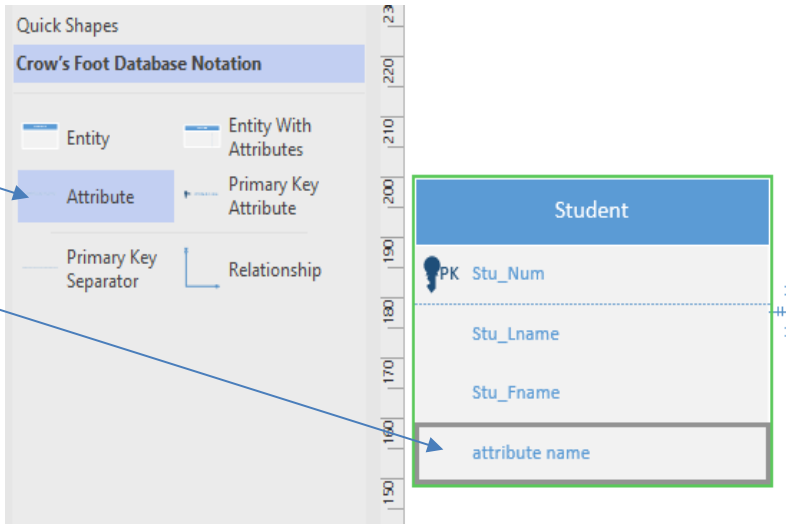
Part a

a. Apply relational constructs to covert conceptual model into a logical model.

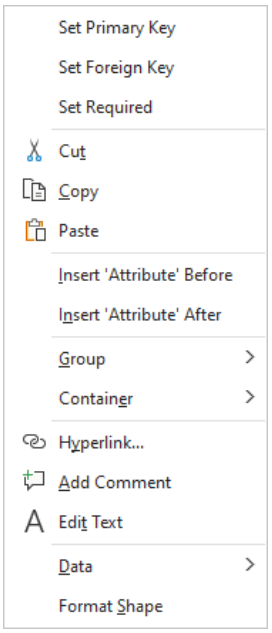
In Tutorial 1B, we created a conceptual model for Tiny College as an ERD using Visio. In Tutorial2 (Part A), we will use the same model and add more information (based on relational constructs) to arrive at a logical model. We will modify the diagram by adding further information (e.g., attributes i.e., columns, data types, etc.) or (if necessary) by altering already added information.

i. Open TinyCollege model (e.g., TinyCollegeConceptual.vsdx) created in Tutorial 1B. Before starting to work further, keep a backup of this file that presently contains the conceptual model. Give an appropriate name (e.g., TinyCollegeLogical.vsdx) to the current working file for the logical model.

ii. Add appropriate attributes (columns) to the entities. Use meaningful and descriptive names to define these attributes. Also, use consistent naming style for enhanced readability. To add more attributes (by default three are given) to an entity, drag and drop Attribute icon at the entity (e.g., A new attribute placeholder has been added to the Student entity). Alternatively, to add more attributes, use the option of Insert 'Attribute' (Before or After) from the given context related menu (to get this menu select an attribute within entity and then right-click).

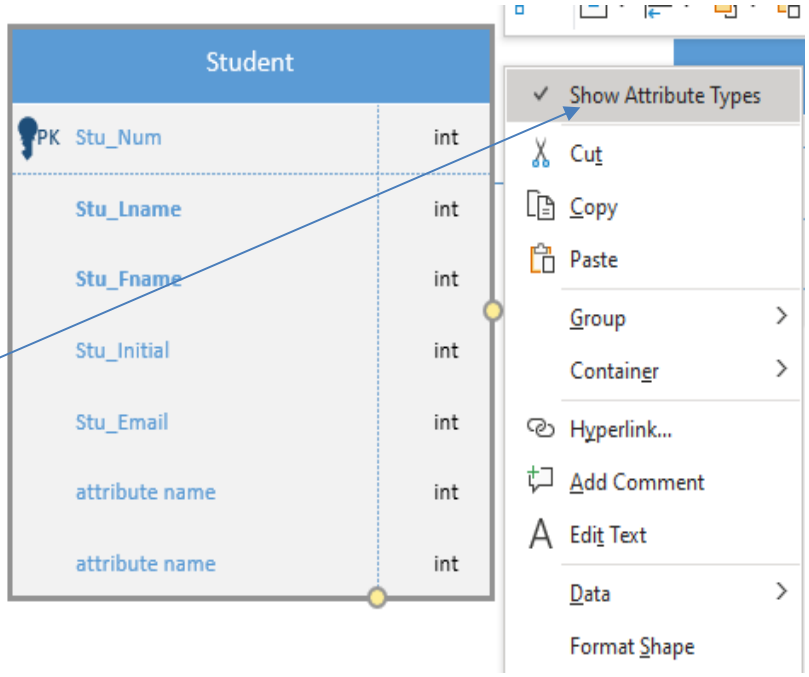


For the required (non-optional) attributes, select the **Set Required** option from this menu. For example, Student first name and last name values are always needed (i.e., required) for the Student entity. Therefore, both attributes need to be flagged as Set Required. On the contrary, Student middle initial (Stu_Initial) is optional (does not always have value) therefore does not need to be flagged as Set Required). This makes the required attributes appear in **bold**.



For each entity, check for functional dependency and set appropriate attributes(s) as Primary Key(s) by selecting the option Set Primary Key (e.g., Stu_Num).

iii. Explore the context-related menu by selecting and r-clicking the entity.



Normally you would select appropriate Data Type for each attribute (e.g., Number, Text with Fixed or Variable Length, etc.) from a given list. However, this option is not available with Visio 2019. When 'Show Attribute Types' option for the entity is used then by default 'int' data types are displayed for all the attributes. Change the attribute data types by typing appropriate data types (select the data type int and edit text). Also, could not find the option to change the attribute (data) sizes (e.g., Stu_Lname should be varchar (15)). So just, type the sizes manually. All this information is not recorded properly as meta-data; but serves our purpose at this stage.

The Student entity here is shown to have four attributes with Stu_Num as the PK. All the attributes except Stu_Initial (which is optional) are set to be Required (these become bold). Stu_Num being a PK is required (by definition) but has not been changed to required using the menu. But if you wish you could do.

iv. Complete your ERD by adding similar detailed information for all the entities.

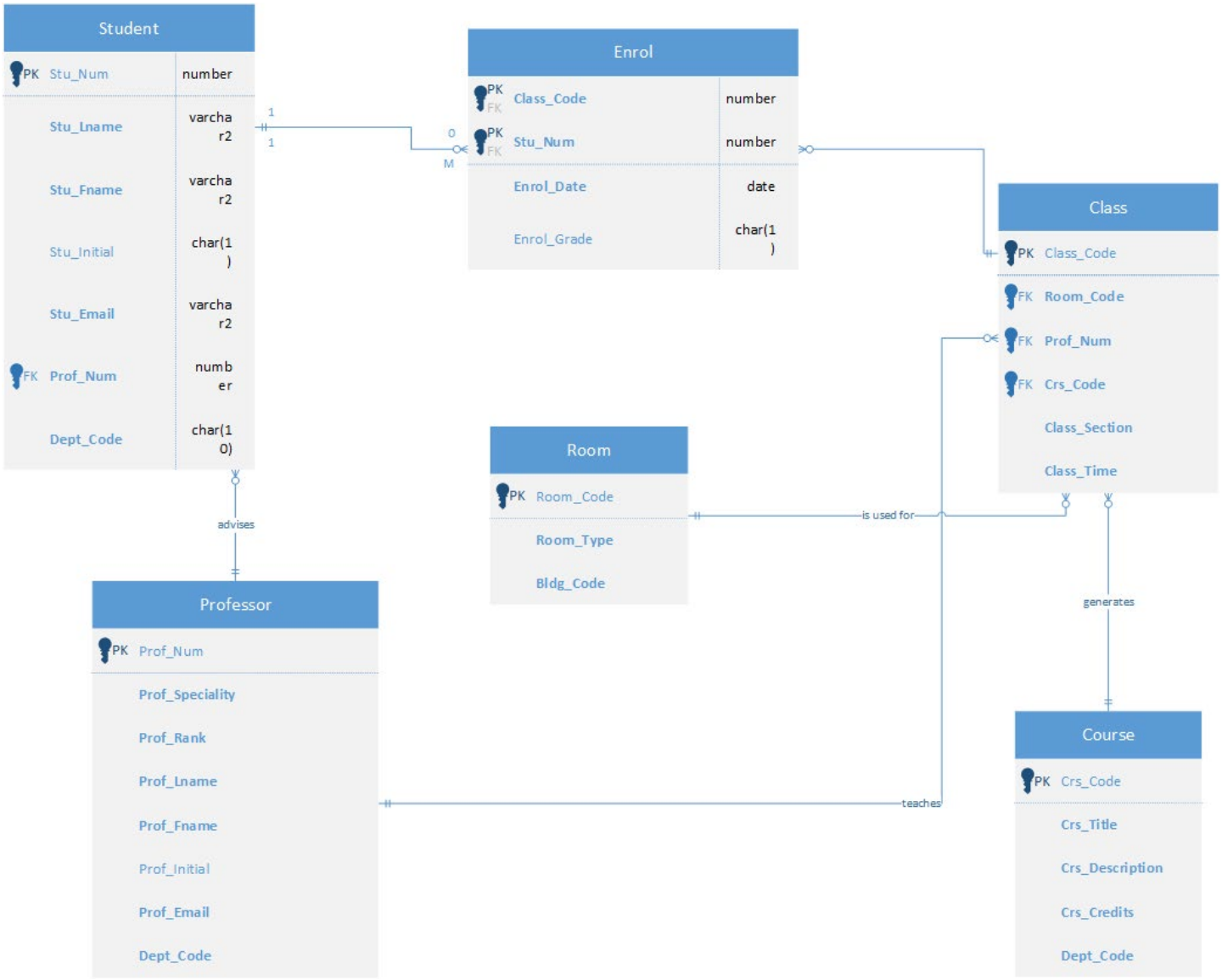
v. Modifications can also take the form of changing or including some missing associations. Add any the missing associations (e.g., association between Professor and Student to store information regarding which Professor advises a Student).
Now add new or modify existing association names ('advises', 'teaches', etc.). Remember association names should read from 1 to M.

Note: Obvious associations do not always need to have names; add an association name only if it adds value to the readability of the diagram.

vi. Modify or add the appropriate multiplicities (i.e., cardinalities) on both ends of the association (sometimes these could be specific numbers e.g., 5...50; although in our example you would not need to). Usually depending on the multiplicities on the association, the foreign keys should be flagged automatically. However, with Visio 2016 onwards (including 2019) you will have to do this manually (select the attribute and set that attribute to be '(Set) Foreign Key' from the context related menu). For example, Class entity has three foreign keys (see the following diagram).

Note: Always check your diagram for correctness and make appropriate changes whenever necessary.

Modified Data Model Diagram for Tiny College



Note: At this stage, do not pay too much attention to whether an association should be shown as a dashed (i.e., non-identifying) or a solid line (identifying). This concept is part of the chapter 4 and is covered in the following lecture/tutorial. From Visio 2013 onwards, a relationship is flagged as a strong relationship (identifying) by selecting a relationship and then checking **Set Identifying** from the context related menu.

The diagram is not displaying (most) data types (to save space) for most of the entities and most data types are not displaying sizes (except for char). Visio 2019 does not record this information properly (as meta data) hence, left like this for these reasons for now.

Part b

b. **SQL based Tutorial 2 – (Relational Set Operators)**

i. Download the **Chapter3_StoreCO-ORA** script file from week2 of the course Stream site.

ii. Run this script file under SQL developer to create and populate three tables called Employee, Region, and Store.
If your account already has any tables with the same names, then either drop or rename those tables for this script to run successfully.

Notes (read these before you attempt iii. below):

- No constraints exist for the tables in the given script file. For some of the questions below you must add asked constraints (e.g., add the PKs). Make the required changes to the code in notepad. Before you run the revised code for a table, you will need to the drop table, as the table now already exists in your Oracle a/c. You can either use the drop table statement (e.g., drop table employee cascade constraints;) or use GUI (i.e., select a table under your connection and r-click and select Table→Drop from the context related menu) to drop a table.

- Code for most of the constraints can be found from the lecture slides 21-24 (lecture 2).

- For (iii c.) below read the following information about NUMBER data types:
For numeric columns, you can specify column as:
column_name NUMBER

  Optionally, you can specify a **precision** (total number of digits) and **scale** (number of digits to the right of the decimal point) as:
column_name NUMBER (precision, scale)

- Give meaningful names to the constraints when appropriate.

iii. Now attempt the following questions based on this database:

a. Identify primary keys for each of the given three tables.
b. Change the given SQL scripts to mark the identified columns as primary keys.
c. Format the column STORE_YTD_SALES to allow it to accept the values with **two** decimal places e.g., 2342200.55.
d. Add a constraint indicating that the column REGION_DESCRIPT cannot be null.
e. Add a constraint that STORE_YTD_SALES value cannot be less than 1000.
f. Write a simple SQL statement using the UNION Operator between STORE and EMPLOYEE tables.

g. Write a SQL statement to display the names of employees who are store managers. The complete employee's name (i.e., employee title, first name, last name) and the store name should both be listed. Which join will you use?
h. Write a SQL statement to display the names of the employees (i.e., employee title, first name, last name) who do not manage stores. Which join will you use?
i. How many rows will be returned when a Cartesian product is taken between STORE and REGION? Also, check the result set with the COUNT operator.
j. Show that INTERSECT and EQUIJOIN operators will return the same result set.

Part c

c. Analyse some relational operators using the script provided.

i. Create and populate three tables named t1, t2 and t3 using the following script:

```
prompt "my joins"
create table table1 (
  col1 number);
create table table2 (
  col2 number);
create table table3 (
  col3 number);

insert into table1 values (1);
insert into table1 values (2);
insert into table1 values (3);

insert into table2 values (2);
insert into table2 values (3);
insert into table2 values (4);

insert into table3 values (2);
insert into table3 values (3);
insert into table3 values (5);

commit;
```

ii. Analyse the relational operators used in each of the following SQL statements. Execute these statements and look at the returned query result sets to understand how these operators work.

a. select count(*) from table1, table2, table3;

b. select count(*) from table1, table2;

c.
```
select col1 from table1
union
select col2 from table2;
```

d.
```
select col1 from table1
union all
select col2 from table2;
```

e.
```
select table1.col1, table2.col2
from table1 join table2
on (col1 = col2);
```

f.
```
select table1.col1, table2.col2
from table1, table2
where col1 = col2;
```

g.
```
select col1 from table1
union all
select col2 from table2
minus
select col3 from table3;
```

h.
```
select col1 from table1
union all
(select col2 from table2
minus
select col3 from table3);
```